Feature-Enhanced Machine Learning Models for Credit Card Fraud Detection

Muhammad Haseeb Khan
202385638

Master of Data Science (MDSc)

Capstone Project

Supervised By
Dr. Asokan Mulayath Variyath & Dr. Jahrul Alam
Department of Mathematics and Statistics,
Memorial University of Newfoundland

Table of Contents

1. Abstract

Credit card fraud positions a significant and growing threat to the global financial system with fraudulent transactions accounting for substantial economic losses each year. In this capstone project, four machine-learning algorithms were tested regarding real-time fraud detection in terms of a large publicly available synthetic dataset with 1.29 million training and 0.56 million testing transactions. A logistic regression baseline (Model 1) gives a baseline estimate establishes a reference performance at highly imbalanced (~ 0.6 percent fraud) dataset with moderate recall (46.6%) at high precision (44.9%) and an ROC AUC of 0.93. By utilizing this, Model 2 is a variant of Random Forest classifier that added the geolocation distance feature to it, its recall was 72.1%, it had a precision of 74.3% as well as an ROC AUC of 0.981. The best balance can be obtained in Model 3 using XGBoost with category-level encoding of the fraud-rates which results in 65.4% recall, 85% precision and ROC AUC of 0.98. Lastly, Model 4 augments a Random Forest with merchant-risk scores and transaction-velocity features and performs with a recall of 75.1% at 78.6% precision and ROC AUC of 0.990, so it is worth consideration when it is critical to detect all cases of fraud. A comparison of confusion matrix and precision-recall trade-off by domain-inspired features noted that the introduction of features leads to significant boost in the performance of detection compared with the baseline. These observations suggest that a meaningful combination of features such as transaction distance, historic fraud propensity and behavior velocity can be instrumentally composed to detect fraud with considerate false positives. We conclude by discussing the operational implications, remaining challenges such as adapting to novel fraud patterns and reducing false alarms and avenues for future work, including hybrid ensembles, sequence-based deep learning, and cost-sensitive threshold optimization.

2. Introduction

Cashless transactions have become the order of the day in the digital economy and billions of purchases are made using credit cards each year. This is the trend of rising electronic payments that has also come with an equally increasing incidence of fraud, which has caused massive financial losses and a lot of insecurity to consumers concerning payment systems. Recent estimates suggest that worldwide credit and debit card fraud losses rose to more than 34.4 billion dollars in the year 2022 and are anticipated to increase even further, and the cumulative loss over the coming decade may total more than 400 billion dollars of losses. Though financial institutions and merchants also incur losses, unauthorized charges typically carry no liability on the part of the cardholder (due to regulations and network policies). In addition to direct financial effects, fraud may damage trust in online payment systems and add up huge costs of operations to the banks (e.g. investigation and customer support).

An effective fraud detection mechanism is crucial to mitigate these problems. It needs to be a high-quality but at the same time efficient fraudulent transactions detector system, either in real-time or near to it to ensure the minimal loss of money with minimal false alarms, which may affect legitimate consumers. High false-positive rates (genuine transactions incorrectly flagged as fraud) may result in poor customer experience, e.g., customers being incorrectly declined on valid purchases; this can cause customers to churn off to the competitors. Hence, the problem of fraud detection can be viewed as a rare-event classification problem and that this problem requires high-recall (to capture as many frauds as possible) and high precision (not to bother too many legitimate users) at the same time, which is challenging to strike.

Earlier strategies of detecting frauds were based on rule-based systems defined by experts and manual inspection. These may be constraints such as limits to the amount of a transaction, blacklists of lost or stolen cards, or straightforward velocity checks (i.e. limiting a card after a few transactions within a short time). Although this kind of rule is interpretable and can detect known patterns of fraud, it is hard to scale to new and evolving fraud tactics as well as produce numerous positive or negative false results. Machine Learning (ML) methods have become popular in the detection of fraud in recent years because they are capable of finding patterns in the data automatically by learning them. Supervised learning models could be trained to identify which historical transactions were fraudulent or otherwise and may even be able to identify previously undetectable complex and subtle patterns that humans or more rudimentary rules may overlook. Nonetheless, ML has some challenges when applied to credit card fraud detection:

1). Extreme class imbalance, meaning that fraudulent transactions are generally less than 1 percent of all transactions, a condition that causes standard classifiers to be biased in favor of the majority (non-fraud) class.

2). Advances and adaptation in the behavior of fraudsters as they continuously find new, more effective, tactics and the models must be able to adapt as well.

3). Real-time performance is required since the decisions need to be made within seconds or faster, which means that the models must run efficiently.

4). Interpretability and trust as financial institutions often require explanations for why a transaction was flagged, for regulatory and customer service reasons.

In this project, we overcome these problems by using ML algorithms and do a thorough feature engineering and evaluation. We use publicly available synthetic credit card transaction data to train and test several models, including a simple logistic regression model as well as more complex ones such as ensemble-based methods. The object of the discussion is the argument that domain knowledge can be embedded in the modeling process in the form of engineered features that record known fraud risk

factors, including geographic distance, merchant or category riskiness, and transaction frequency patterns, and consequently enhance detection performance dramatically. We also adopt strategies to deal with class imbalance and measure the models using the proper metrics instead of the mere accuracy (which can be artificially high in the imbalanced scenario).

2.1. Motivation

The motivation behind this project is the fact that credit card fraud is on the rise in a cashless society and this necessitates combating it. With the increase in digital payments, fraudsters have increased in the levels of sophistication with regards to exploitation of vulnerabilities. Credit card fraud leads not only to direct loss but also to the loss of the reputation of merchants and financial institutions. Publicized data breaches and prevalence of hacked card data in black markets have made the attempts of fraud more frequent and internationally organized. According to a survey conducted by PwC in 2022, 56% of the organizations worldwide have faced fraud or other economic crime within 24 months, and the most common type of it is banking or payment fraud. More than 390,000 cases of credit card fraud were reported to the authorities in the United States alone in the year 2020. Those figures make it clear that fraud is not a rare phenomenon on an absolute basis, with tens of millions of fraudulent transactions taking place each year, although any given entity may experience only a negligible percentage of its transactions to be reported as fraudulent.

- **Protecting Businesses and Customers:** An effective fraud detection system will protect merchants and consumers against having unauthorized or unconfirmed purchases and protect business and financial institutions against the secondary expenses of fraud (chargeback, investigation, fees). It also helps to protect customer trust in digital payment channels by reducing the losses to fraud.
- **High Stakes of Errors:** Missed fraudulent transaction costs money and can also attract more abuse whereas false decline of genuine purchase erodes customer experience as well as revenue generation. It aims to maximize the precision recall trade-off, to detect more fraud but without an equal increase in false alarms.
- **Leveraging Data and ML Advances:** We now have access to millions of (or perhaps even billions of) transactions and modern computing power to rely on, so we can go beyond hard and fast rules and instead train models (ensemble trees, deep learning) that can learn complex fraud patterns. The comparisons between logistic regression, random forests, and gradient boosting on a dataset of 1.8 million transactions can attest to the value and performance of these machine learning methods and their use to find answers.

- **Academic and Educational Perspective:** This capstone project not only deals with cross-disciplinary application of data science and finance to cybersecurity-related problems, but also with technical and domain knowledge, as it involves a lot of theory, tested in the context of a high-impact, real world problem. In addition to high predictive performance, it also sforks to generalize the best practices in feature engineering and model selection to general anomaly detection tasks.

2.2. Scope of the Study

The context of the given study is limited to areas of supervised machine learning involving detection of credit card transaction fraud, with predefined data of the historical transactions. The key points of the scope include:

- **Data Modality:** We focus on structured tabular data (transactions records containing various attributes). Other modalities such as phone/device meta data, unstructured text, or picture of cards are not in our scope. We also do not explicitly address the network-level fraud detection (e.g. by looking at the social networks of fraudsters) or other fraud types such as application fraud.
- **Algorithms Covered:** We talk about conventional and ensemble algorithms of machine learning (logistic regression, random forest, XGBoost gradient boosting) and not deep learning models. It is due to the fact that we have tabular and comparatively low-dimensional features, and tree-based ensembles are considered to work on these data well.
- **Feature Engineering:** Our major focus and contribution to this project is related to feature engineering. Surrounding the domain of deriving specific more features with respect to the raw data which encodes the domain knowledge (point distance, risk encoding, velocity, and so on). As far as we restrict our engineered features to the features computable based on the available data fields somehow (e.g., we have location coordinates that we can compute distance, timestamps to compute velocity). We are not utilizing any external data (e.g. existence of fraud scores in other sources or social media data).
- **Evaluation Metrics:** We pay particular attention to the evaluation metrics that can be used in imbalanced classification: confusion matrix breakdown, Precision, Recall, F1-score of positive (fraud) class, and ROC AUC. We can report accuracy for completeness but less importance is given to it because of class imbalance. The effect on cost-sensitive metrics or dollar value of fraud doesn't appear to be explicitly modeled (the dataset does not give different costs per transaction except marking them as fraud/not fraud).

- **Temporal Train-Test Split:** The datasets are already split into two parts: a training (fraudTrain.csv) and test set (fraudTest.csv). We assume that such a division resembles a real scenario and we will train the dataset on fraudTrain.csv dataset and test it on fraudTest.csv dataset. This scope covers following this split without cross-validation or random splits, to consider possible temporal dependencies (fraud behavior may drift with time). We observe that the tuning of hyperparameters (random forest and XGBoost) is made using the training data (which can be via internal cross-validation) and that the final performance is the one on the given test set, to mimic a real deployment.
- **Real-Time Considerations:** Although we refer to the real-time requirement to detect, implementing a model or streaming data is not within the scope of the project. We carry out an offline analysis. Our models do not directly measure latency or throughput, although we talk about it qualitatively, indicating that our selected models (tree ensembles in particular) can be tuned to perform fast in practice.
- **Domains and Generalizability:** The study itself focuses on credit card fraud in the e-commerce/banking domain, but we take in our discussion the prospects of how the methods can be extended to other related domains (insurance fraud, health care billing fraud etc.). But we do not assess our models on other databases or fraud categories.
- **Ethical issues and Privacy:** Ethical and privacy issues are not directly related to the scope of any technical work, as the data is synthetic with a relative degree of anonymity. We do commonly note that models may have encoded biases or may even require special interpretation to prevent the imposition of unwarranted bias on groups, but we are out of scope examining this problem.

In general, the scope is an end-to-end applied machine learning project of a certain fraud dataset, data understanding and preparation, model development and evaluation, and interpretation of the model, but it does not stretch into production deployment or multi-modal data integration. This way by ensuring that the scope is defined, we are making the study manageable and meanwhile the conclusions arrived are well backed up with the experiments that were carried out.

2.3. Applications in Various Domains

The use of machine learning and data analytics is essential to fraud detection in many other cases besides credit card purchases. Although this project is about fraud in relation to credit card payment, it has principles and methods which can be applied across many fields:

- **Banking and Credit Fraud:** In the banking field beyond the transaction fraud in credit cards, there is also the problem of account takeover (where criminals can set up an account of a bank without its approval), check fraud and wire fraud. Groups like banks further keep track of activities in bank accounts or loan bets. Our learning techniques (classification, anomaly detection, behavioral features) can directly apply in those cases, as it is also about separating the legitimate vs. fraudulent behavior based on the records of the transactions.
- **Insurance Fraud:** Auto, health or property companies deal with insurance fraud. Identification of these is often conducted by examining claim particulars, the history of the claimant and on occasion, third-party information. Although the data is of a different (text descriptions, claim amount, claimant profile) structure, machine learning models are applied to predict the probability of a claim being fraudulent. Examples of a feature like our risk encodings would be, e.g. the risk given by some specific medical providers or garages (this would be our merchant risk) or the number of claims caused by one of our customers (our transaction velocity).
- **Healthcare and Medicare Fraud:** Under healthcare billing, fraud may be practiced when an individual charges for services that had not been offered or done upcoding of procedures. Models are used in detection of anomalous billing patterns of providers that are driven by data. This reminds us of how we have noticed anomalous spending transactions. Asked about the meaning of features such as distance, features may correspond to the distances between a patient home and the services location (to cover improbable patient visits, which is the same as capturing improbable purchase locations). Risk encoding may also be employed to allocate a risk score to each healthcare provider depending on the history of fraudulent billing cases like merchant risk.
- **E-commerce and Retail Fraud:** Online retailers usually fall victim to fraud that occurs mostly in the non-banking world through order fraud (using stolen cards or identities) and refund fraud. Fraud detection checkout in many e-commerce systems is based on ML. These are frequently implemented based on such characteristics as device fingerprint (existence of the same device on multiple cards), distance between an IP and shipping address, transaction velocity originating through the same account etc. These have a direct parallel to features in our credit card fraud problem (IP/location is equivalent to our geolocation feature; velocity of orders is equivalent to transaction velocity). We might, therefore, use our method to e-commerce payment data with few modifications, as it also represents a two-class division of transactions.
- **Internal Corporate Fraud:** Companies have also come to watch the internal transaction to defraud as well (payroll fraud, procurement fraud). The data may be corporate expense reports or payment ledgers, and anomaly detection can be used to signal abnormal transactions (e.g. an employee has an unusual

frequency or level of expenses claimed). Although used in a different domain, methods such as isolation forests or clustering may be relevant and so may supervised models in case one has labeled examples of previous internal fraud.

- **Telecommunications Fraud:** Telecom companies often face fraud through schemes like fraudulent calls or subscription fraud. An example is identifying fraudulent phone calls (whether within VoIP communication or cellular network) with the help of phone durations, origin/ destination pairs, call frequencies, once more comparable to a transaction amount, merchant, and velocity in monetary fraud. Deceptive SIM cloning or subscription can also be detected based on geographic discrepancies (a person using the phone in two distant locations within a small window of time, like a credit card being used miles away from his or her home within a few minutes).

In short, the methodologies at hand in this credit card fraud project - the logistic regression and ensemble models with custom features - capture how fraud detection is carried out in most realms. Its implementation and features may vary but the core control panel of data science (finding rare anomalies with high confidence) is generic in fraud prevention. That reinforces the more general applicability of our work: progress or discoveries in one area (such as credit card fraud) can frequently be converted into better fraud detection in other areas, whether insurance, e-commerce, or otherwise.

3. Literature Review

3.1. Existing Methods: Overview and Relevance of Fraud Detection

The issue of credit card fraud detection has been explored over several decades and there has been a wide array of methods that have been suggested in literature. One can divide the approaches into four main groups: (1) Knowledge-based systems and expert rules, (2) Traditional statistical methods, (3) Classical machine learning algorithms, and (4) Recent advanced techniques (ensemble methods, deep learning and so on). We provide an overview of all of them with emphasis on those the most relevant to our approach.

Rule-Based Systems: In the early days, sets of rules created by the experts in the form of if-else statements could be used to detect fraud in banks initially. An example may be rules such as those used to flag transactions that are above a particular amount, transactions that took place a distant state away from the cardholder, or a quick succession of transactions within a short period. These regulations are simple to understand and use. Nevertheless, they do not detect new patterns in frauds well and tend to have high false positive rates. People who want to cheat can also get around

established limiting measures by operating in areas that are just below the set limits. Static rules could not keep pace with the changing nature of fraud (such as coordinated low-value fraud or fraud using newly emerging channels such as online payments). However, certain rule-based logic (such as velocity rules) is still found in use as a subsystem of other systems, frequently to engage instant blocking or multi-factor authentication.

Anomaly Detection and Unsupervised Methods: Since fraud is rare, a naive approach is used to identify any outlier transaction that does not seem to be ordinary. Unsupervised techniques such as clustering and outlier detection by means of distance have been considered. As an example, K-Means clustering can cluster transactions and find small clusters of outliers and these might be fraud. Similarly, distance-based approaches or density estimation may raise the alarm over transactions that are on the outskirts of normal spending patterns of a user or the population. Other studies suggested hybrid schemes that incorporate clustering with other algorithms to detect fraud. The unsupervised approaches are attractive since they do not need labeled instances of fraud, which is beneficial when there is limited or when novel types of fraud are to be detected which were not available in training. But they have limited power: not every fraud occurs as a statistical outlier (which may mimic legitimate behaviour to pass undetected), nor every outlier a fraud (which might just be a straightforward outlier but not a fraud). Our project mainly utilizes supervised learning, although we implicitly make use of an outlier-type feature, where we compute the distance to home (geographic distance) as an approximation of how unusual the location of certain transactions is concerning a particular user.

Traditional ML Algorithms: During the 1990s and 2000s, logistic regression, decision trees, and neural networks were used by researchers who applied them to fraud detection. Statistical generalized linear models (such as logistic regression) provided a more explicit way to sum a linear combination of each feature and give a score of fraud. To a certain extent, these models can be easily implemented and interpreted (coefficients signify risk factors). But they might not be able to capture non-linear inter-feature interactions and patterns. Decision trees also became popular, since they are inherently natural in nonlinear interaction and they could be trivially interpreted as a set of rules. One decision tree may not be quite accurate because of overfitting or instability, but they helped to pave the way towards more complex tree-ensemble methods.

Ensemble Methods: Ensemble learning has been very useful in terms of detecting fraud. It is found that Random Forests (as an ensemble of numerous decision trees) tend to perform best and Gradient Boosting Machines (such as XGBoost, LightGBM) are widely reported as close to the best. These techniques lower variance by aggregating several trees and are able to estimate complex relationships. As an example, Random

Forest turned out to be better than many other classifiers at a recent study showing high scores of accuracy (99%) and recall on a credit card fraud data set. Explaining the high performance of tree ensembles, their capability to learn patterns involving interactions (e.g. a transaction occurring at night and far away and a high amount may all be indicative of fraud together) and its robustness to noisy features. The gradient boosting tool such as XGBoost are advantaged in most applications since they provide a superior approach to class imbalance (through objective weighting) and overfitting through regularization. Through findings of ideas such as boosting having highly beneficial precision-recall tradeoffs, due to their ability to adapt to minority class data, our Model 3 is based on XGBoost.

Data Imbalance Techniques: An essential point in literature is the handling of the class imbalance. There are a few methods: oversampling the minority group or undersampling the majority, synthetic data generation (SMOTE- Synthetic Minority Over-sampling Technique, generating artificial representatives of the minority), and cost-sensitive learning (weighting fraud misclassification more in the loss). According to numerous studies, the use of such techniques as SMOTE with ensemble classifiers has shown an improvement in recall. These are the techniques that we took into consideration in our approach. Finally, we did not implement SMOTE in any of our final models, as creating artificial patterns with our engineered features and feature construction was likely to overfit; thus, we simply used tuning of the different models (such as the inbuilt feature for imbalance control in XGBoost) alongside adjusting the evaluation thresholds. However, knowing these methods is significant and we refer to them as possible advances.

Deep Learning: More recently, fraud detection has been implemented using deep learning. Researchers have utilized feed-forward neural networks, autoencoders as anomaly detectors, recurrent neural networks (to model sequential spending patterns), graph neural networks (to find rings of fraud across networks of merchants and cards). Deep learning has the capability of automatically discovering feature representations and is also able to identify very complicated patterns. Nevertheless, there is limited success reported in the literature: some deep models underperform well-tuned simpler models on tabular fraud data. The likely reason is that the organized nature of the features (amount, location, etc.) does not seem to necessitate the advantages of representation learning much of deep networks and that the small amount of fraud data available to train the large networks. With that said, we have successful instances such as exemplifying autoencoders as a method of encoding normal behaviour and detecting anomalies or that of graph networks to detect suspicious connections (e.g., various cards indicating abnormal behaviour in the same device or IP could likely be organized criminal activity). Practically, tree ensembles are a staple because they are efficient and interpretable. In our project, like the rest of the industry, our efforts are concentrated upon tree-based models over the deep networks of available data, and yet it is worth

recognizing that deep learning is a potential path to be pursued particularly in the case of alongside other sources of data or in a real-time adaptive system.

Hybrid and Other Approaches: There is some literature on hybrid models integrating unsupervised and supervised methods e.g. employing clustering to find suspicious groups and then feeding those as feature to a classifier. There are others who utilize meta-learning or the stacking where the output of a series of various models is fused. Also, transaction sequence analysis research makes use of Hidden Markov Models or sequence mining to identify when a sequence of transactions would be unexpected for a certain user. A second branch of study utilizes Bayesian learning and probabilistic models and involves viewing the detection of fraud as updating the likelihood of fraud based on newly available evidence based on prior information. Such approaches are able to deal with uncertainty and include prior knowledge as expert knowledge. Overall, the literature depicts an obvious development: the transition to automation and large amounts of data. Specifically, ensemble ML methods have gained remarkable popularity and achieved good results, placing highly in competitions and real-life application. The connections to our work are direct (and in the case of sampling and cost-sensitive evaluation, direct, although we have not made use of SMOTE, we did experiment with oversampling). We make use of logistic regression as a baseline (this relates to the prior work done on statistical models), of random forests and XGBoost as state-of-the-art classifiers (this is related to the success of ensemble learning), and we have made use of concepts such as sampling (though we did not consider using SMOTE, we did consider oversampling). The importance of critical assessment in the literature is also relevant: in most cases (and this is also true in this project) accuracy is irrelevant in these datasets and precision/recall or AUC take center stage.

3.2. Key Prior Works

To situate our methodology, we note some of the preceding literature and references that our study informs or complements:

- **Dal Pozzolo et al. (2015) -** A monumental study on credit card fraud (on a dataset of cards of a European country) that highlighted the deployment of Balanced Random Forests and Balanced cascades and the analysis of precision-recall curves. They demonstrated that in the case of extreme imbalance, adjustment of classifier thresholds and emphasis on recall vs false positive trade-offs is paramount. Our choice of precision/recall and ensemble procedures is supported by their work.
- **Bahnsen et al. (2016) -** Presented cost-sensitive credit card fraud detection. They put a dollar cost on false negatives (the fraud amount lost) and on false positives (customer insult cost). They trained cost-sensitive classifiers using a

cost matrix. Although we will not necessarily employ cost matrices, this concept comes alive in our solutions method as we speak of threshold tuning (subconsciously balancing between recall and precision). It has also informed us of the concept of risk/category or risk/merchant scores in our feature engineering as well which is basically a quantification of the historical costliness (riskiness) of a category or merchant.

- **Whitrow et al. (2009)** - One of the earliest to suggest behavioral features for fraud, such as the number of transactions in the last 24 hours (velocity) and expenses in time periods. They have detected that such features are very effective in improving the detection against the use of just static transaction attributes. This had a direct bearing on our inclusion of the transaction velocity at Model 4. Our methodology could be regarded as one that stretches this concept further by computing distance and risk attributes as well.
- **Jha, Guillen, and Westland (2012)** - it examined a Bayesian belief network and an outlier-based approach to detecting fraud. They also cited as helpful the large number of frauds happening in bursts (which once more supports the idea of the velocity features) and that spatial distance can be a giveaway (a card utilized nearly concurrently in various cities is dubious). Their conclusion included an early indication that a feature such as distance to last transaction is valuable, which in our case, we use a simplified version calculating the distance between the home address of the cardholder to purchase address.
- **Recent Ensemble Studies (2022-2024)** - A number of recent articles, such as a 2024 IEEE Access article by Jemai et al., have compared multiple ensemble classifiers on both real fraud datasets and simulated fraud data. They may discover that high equilibrium AUC (>0.98) on the popular Kaggle European dataset does not translate to low or at least different AUC values on simulated data. Interestingly, Jemai et al. reported that deterministic phenomena in simulated data (such as, the Sparkov dataset we use in this project) can cause models to overfit those phenomena and subsequently poorly generalize to new simulated logic. The insight is as follows: it cautions us that our model might be picking up quirks of the simulated data that wouldn't hold in real data. We address this by focusing on more general features (e.g., "distance" and "velocity" would matter in real scenarios too, not just simulation) and by testing on a separate simulated year to ensure at least temporal generalization.
- **Deep Learning and Hybrid Approaches:** Remarkably, a 2019 article by Fiore et al. used an Autoencoder + Random Forest hybrid, with an autoencoder training a lower dimensional representation of transactions and RF being used to differentiate between fraud and not. It improved a bit, and this suggests that non-linear feature learning can assist. In the meantime, 2023 research by Roy et al. attempted to use graph neural networks to identify cardholder-merchant social communities to identify fraud communities. Those are state-of-the art as well as

more cumbersome to implement. We remain with more conventional techniques; nonetheless, the high performance of graph-based techniques particularly indicates that risk-merchant representation features (as is the case in Model 4), are on the right path because the graph model is very much learning which merchants tend to engage in fraudulent practices.

Overall, previous studies stress that the combination of robust classifiers (ensembles) and smart features (particularly time and aggregate features) together with calibration to the business context (cost of errors) are keys in such endeavors. We include such lessons as applying ensemble models, including such features as distance and velocity against what previous studies indicate, and measuring with domain-relevant metrics. Our methods are innovative mainly in the way we incorporate certain tailored characteristics of the particular algorithms with known characteristics and compare their advantages incrementally on a full-scale dataset. Although we do not present a fundamentally new algorithm, we merge ideas collected in the literature into a synthesized modelling strategy and present a comprehensive case study of this being effective on a modern and large fraud dataset.

3.3. Understanding Model 2, Model 3, and Model 4

As stated before, Model 1 is a baseline Logistic Regression with the simplistic features available in the data (after some clearing). Our initial point is, we believe, a representative of some common method, which did not include any additional adjustments. Then we developed Model 2, Model 3 and Model 4 with the addition of more advanced techniques and features to them. In this section we describe the main concepts of each of these models i.e. what they are and why did we select them, their expected benefits as well as their possible difficulties.

Model 2: Random Forest with Distance Feature. The main idea of Model 2 is the fact that it adds into the fraud detection model one feature representing geographic distance and applies an effective ensemble classifier (Random Forest) to manage relationships and non-linear patterns. We calculate the distance between the location of the transaction and the home location of the cardholder using Haversine distance formula. The hypothesis being postulated is that fraudulent transactions are mostly conducted in places that are distant to the legitimate owner of the card, or the area of activity. As an illustration, when a card owner (say in New York) makes a sudden transaction in California or other countries within a short observation period, the transaction is most likely to be fake unless the card owner is travelling (which is not that common). This often plays into the rules at many banks (“card-present transaction in a country that the user never visited generates an alert”). Encoding distance provides the model with a quantitative scale of this anomaly. Random Forest was selected as the

classifier as a model can easily consider an extra feature and model interaction between distance and other features (such as amount or time of the day) effectively. Random Forest is composed of a combination of decision trees, hence any given tree can be separated based on, say distance > 500 miles and transaction time that is odd to get a high-risk segment. These patterns are then stabilized by the ensemble vote.

Advantages: Model 2 will hopefully pick up the fraud patterns pertaining to location anomalies not possible to pick up in Model 1 that lacked an explicit location feature. Random Forest is also likely to enhance the overall detection because it is more capable compared to logistic regression. It is also strong to irrelevant characteristics, like the distance might not always be a good signal about fraud (cardholders might travel). In that case, the random forest can capture when it matters alongside other pieces of evidence.

Challenges: Adding distance may add false positives of legit transactions who are simply not at home (e.g., someone making a legit trip). Other features may tell the model that unless others say it is not so. Not only that, distance calculation also uses coordinates; our data does give us coordinates but in deployment we need to have that ready. Random Forest, which is more complicated, can overfit without tuning, so that is another problem, particularly with the imbalance (in the case of a lot of trees, variance is minimized). We balance it out by tuning the parameters adequately and checking the feature importance so that it is intuitive.

Model 3: XGBoost with Category Risk Encoding. The major innovation of Model 3 is category risk encoding. One of the features of the dataset is a categorical one, named “category” and describes the type of a merchant (e.g. grocery_pos, gas_transport, etc.). We do not one-hot encode these (which would give us a lot of sparse features) or treat them as labels but rather assign each category a risk score: the historical fraud rate of that category in the training data. In that case, assuming that shopping_net category has 1.75 percent fraud rate and grocery_pos has 0.5 percent then the model is supplied with such numeric values as a feature. This successfully pre-informs the model that online shopping fraud susceptibility is greater as compared to physically going to grocery shopping (most likely to indicate that online transactions are more risky). This transformed feature is then trained with XGBoost classifier among other features. A very high-performance and heterogeneous features-friendly gradient boosting tree algorithm is XGBoost (Extreme Gradient Boosting).

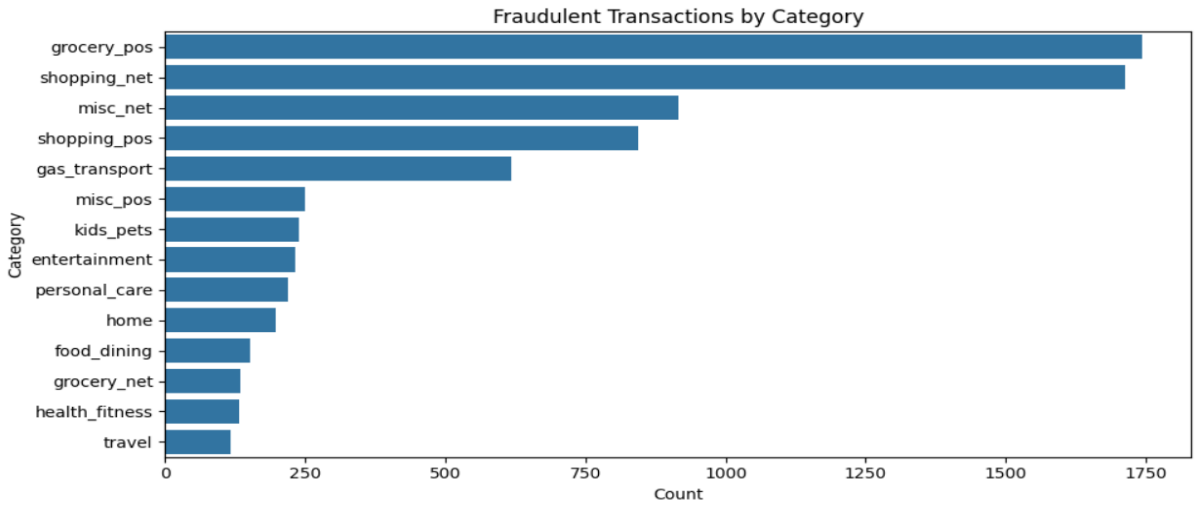

Figure 1: Fraudulent Transactions by Category

Advantages: Category risk encoding is a target encoding that can enhance predictive power by giving the model a direct signal to the fraud probability of a feature value. It can record worldwide trends of frauds that logistic regression or random forest may only learn indirectly. As an example, a XGBoost without risk encoding may at some point learn such a split where: if category == shopping_net then higher risk, except by encoding that to start with as a number, such a split will be harder to use in conjunction with other features that are numeric. Being more expressive in comparison to logistic regression, XGBoost may subsequently use such encodings and other features (e.g., amount, distance, etc.) upon more fine-grained splits. It is also more effective at solving the class imbalance problem because it automatically balances trees in favor of the positive class and that it can optimize AUC or other criterion as appropriate.

Challenges: One of the main issues with target encoding is leakage- unless performed in a careful way, the model itself might essentially learn the occurrence of fraud. We make sure that the category risk is calculated on the training set only and apply such encodings on test to prevent any test information leakage. The other complexity is that rare categories may have risk estimates that are noisy (e.g., when a category only had 10 transactions and 1 was a fraud then 10 * 1 = 10% risk is statistical noise). We responded to this by potentially flattening out the risk encoding or establishing a floor. XGBoost model could be subject to overfitting when the number of trees or the depth is unrestricted, particularly under such encodings, in which case we adjust its hyperparameters (tree depth, learning rate, etc.). Also, XGBoost is not as interpretable as logistic regression, although we can check the feature importance to see whether category risk feature is really driving much (which we would expect, against our hypothesis on category significance with fraud).

Model 4: Random Forest with Merchant Risk & Transaction Velocity. This model is extended by two new features: a merchant risk score (analogous to category risk, but at

the individual merchant level) and a transaction velocity feature capturing the proportion of transactions on the same card in the same recent time period. We calculate the risk of merchants using the historical fraud rate of a particular merchant in the training data (e.g. if the merchant identifiers of a "fraud_Kutch, Hermiston and Farrell" were 20 out of 1000 transactions have been fraudulent, then its risk = 2%). This can find both fraudulent items (scam merchants) and items commonly victimized by fraud. Velocity of transactions is an aspect of behavior of the cardholder: we determine how many transactions (or total amount) this card previously made, say, in the last 24 hours (or last hour) before this transaction. Fraud may be associated with burstiness i.e., a series of transactions during a small period of time before the card is blocked. We take, say, what was called a feature such as count_of_transactions_last_1_hour or last_24h_transaction_count. Model 4 is based on Random Forest as well; admittedly, XGBoost would be equally an acceptable model, but we are interested in knowing whether Random Forest with these properties would compete with XGBoost and simply to use the different types of models.

Advantages: Merchant risk is a refined addition to category risk. Should a particular merchant be breached or has always been malicious (as may be the case with knowingly fraudulent websites or dubious point-of-sale), this will identify transactions at that merchant. Transaction velocity directly tackles both patterns of frauds such as most bought by the attacker in the least time. An authentic card holder will have a lower transaction frequency, but a fraudster, having accessed the card details, may make small purchases with the card and as many purchases as possible without detection. Such velocity characteristics have been observed to be very useful in the fraud detection system. Incorporating them into Model 4 will enable preventing frauds that can get in successfully via Model 3 (examples are when the fraudster loads the card and uses it at a generally safe category/merchant, but does so 10 times within 5 minutes, and so velocity can flag it, but category risk cannot). With Random Forest, which is able to handle these new features naturally, we project that the recall, in particular, will increase as the velocity is mostly effective in letting us know more cases of fraud.

Challenges: The first problem is that merchant risk encoding will have the same possible leakage problem as category encoding, but to an even greater degree since there are many merchants and some are only on train and some are only on test. New merchants in test (which would be unknown risk; we would want to give them some average risk or default). If a fraudulent merchant from train appears, the model might trivially flag all its transactions – great for catching fraud but what if that merchant cleaned up or those frauds were isolated incidents? There are chances that the model might overestimate the risk. We counteract this by considering merchant risk as it is only one characteristic; the Random Forest will take this into account but must have additional evidence. Velocity characteristics involve grouping transactions according to time and they may be difficult to perform in real-time implementation. In our offline

environment, we must produce them thoughtfully to train (upon which we do not look into the future) and test (where we assume that we will process transactions in order). There are velocity features that we created by grouping by card and calculating rolling counts (without counting the current transaction in its own prior history). Otherwise there was a risk of leaking future info (which we prevented by means of chronological computation). There is also the issue of correlated features, which can be a problem where there may be merchant risk, and category risk (as a merchant has a category). It is possible that the model will double-count risk. We verified the existence of multicollinearity; Random Forest is not as sensitive to this as linear models but it may overweight it. This is catered by doing feature analysis, which might involve dropping one of them in case it is redundant. Finally, we kept both because each of them represents a different granularity.

In short, models 2-4 are iterative developments of the base model, adding one or two important concepts: Model 2 adds non-linear modeling and a location-based feature; Model 3 adds advanced encoding of a categorical feature and a more powerful boosting algorithm; model 4 adds behavior over time (velocity) and fine-grained risk at merchant level. Through these we can notice the effect of each of these ideas. It is expected that every newer model would be able to perform better than the previous one although there is the trade-off of complexity.

4. Dataset Description

4.1. Dataset Details

This study will be conducted with the help of the Credit Card Transactions Fraud Detection Dataset provided by Kartik Srinivasan (Kartik2112) at Kaggle. The data is a synthetic dataset constructed by the Sparkov data generator (a tool developed by Brandon Harris) to resemble real credit card transaction behavior over two years. Its simulation consists of deals between the 1st of January 2019 and 31st of December 2020 with the fixed pool of customers and merchants. It is interesting to note that it also contains both legitimate and fraudulent transactions that are marked accordingly and this is critical in supervised learning.

The most important properties of the data include the following:

- **Size and Split:** The data is already divided into training set (fraudTrain.csv) of size **1,296,675 transactions**, and test set (fraudTest.csv) of size **555,719 transactions.** This makes the total number of transactions within the two years to be **1,852,394.** Our guess is that the training set is akin to 2019 and the test set to 2020 (this is corroborated by the fact that 2019 contains more transactions in

the simulation compared to 2020 perhaps due to an increase in the number of accounts with time). This time-split is an emulation of a realistic deployment condition whereby the model has historical data on which it is being trained and future data against which it is being tested. It does not possess any possible lookahead bias in assessment as well.

- **Class Distribution:** The data is **highly imbalanced**. In the training set, only **7,506 transactions (~0.58%) are labeled as fraud**, with the remaining ~1.289 million (99.42%) being legitimate. The test set has a similar fraud rate (around 0.61%). This imbalance – roughly 1 fraud in 172 transactions – reflects real-life rates (though some real datasets are even more skewed, e.g. 0.1% fraud). This presents a challenge for modeling, as discussed. We confirm that there are no other anomaly labels (just a binary fraud indicator is_fraud which is 0 or 1).

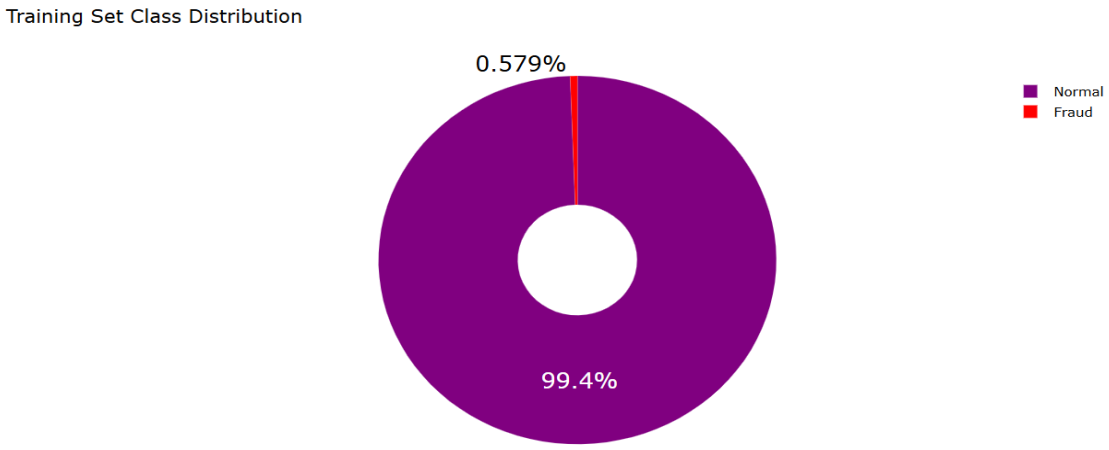

Figure 2: Class Imbalance

- **Features (Columns):** Each transaction record has 23 columns. These can be grouped as:

  Transaction Attributes:

  - trans_date_trans_time: Timestamp of the transaction (when it occurred).
  - amt: Transaction amount in USD.
  - trans_num: A unique transaction ID (probably a hashed identifier).
  - unix_time: Unix timestamp (seconds since epoch) of the transaction time – essentially a numeric version of trans_date_trans_time.

Card Holder Information:

- cc_num: The credit card number (as a unique identifier for the account).
- first and last: Cardholder first and last name (synthetic, but can indicate identity; likely not very useful for modeling except perhaps frequency of same name which isn't relevant here).
- gender: Cardholder gender (M/F).
- dob: Date of birth of the cardholder (thus age can be derived).
- job: The occupation of the cardholder

Card Holder Location:

- street, city, state, zip: The billing address of the cardholder.
- lat, long: Latitude and longitude of the cardholder's home address (derived from zip, presumably).
- city_pop: Population of the city (gives an idea if area is urban or rural).

Merchant Information:

- merchant: Name of the merchant where the transaction took place. The names are synthetic, often containing strings like "fraud_" which are just part of the name (not an indicator; both fraudulent and legit transactions can have merchants with names starting "fraud_" as seen in sample data).
- category: Category of the purchase, e.g., gas_transport, grocery_pos, entertainment, etc. There are both "_pos" (point-of-sale, physical) and "_net" (online) versions for some categories, indicating channel.
- merch_lat, merch_long: Latitude and longitude of the merchant location.

Target Variable:

- is_fraud: 0 for legitimate transactions, 1 for fraudulent ones.

There are no explicit features like "distance_from_home" or "distance_from_last_transaction" given; these have to be derived from the lat/long fields if needed (which we do in our feature engineering). There is also no explicit field

for transaction type (card-present or not) except what we can infer from category naming (_pos vs _net might imply that).

- **Data Quality:** As per the source and preliminary analysis, there are **no missing values.** All the transactions are written down in each field. This simplifies preprocessing (no imputation is done). Since it is simulated, it has consistency and plausibility with things like names, addresses, etc. We anticipated some other more evident incompatibility (inconsistent negative values, or where the date was outside the valid limits) in them but were unable to locate them. The frequencies vary between small (a few dollars) and considerably large (thousands) in terms of the types of purchases.
- **Simulation Aspects:** The data being synthetic has some implications. It was generated using a simulator with certain assumptions (the Sparkov generator). We know from Kaggle description that:
    - There were **1,000 customers** and **800 merchants** simulated.
    - The simulator likely uses a list of merchant names and categories and randomly generates transactions for each customer based on some behavioral patterns (maybe some customers shop more frequently, etc.).
    - Frauds were introduced in the simulation – possibly by randomly selecting certain transactions to mark as fraud with some logic (like a certain fraction in each category, or certain merchants might be fraudulent).
    - The presence of fields like name, job, etc., suggests an effort to mimic real datasets where those might be known, but they may not have strong signals for fraud in a random simulation (though something like age or gender could correlate with amount or usage patterns in subtle ways).

One should be cautious that simulated fraud might not capture all complexities of real fraud (like adaptive adversaries, new categories of fraud, etc.). However, the data is realistic enough to train and evaluate models, and because it's labeled and large-scale, it serves as an excellent testbed for our experiments.

4.2. Data Cleaning

Our data cleaning was quite simple because the data was of a good quality. We explain what was done to pre-process the data into something that can be modeled upon:

- **Dropped Unnecessary Columns:** We had to drop Unnamed: 0 this is just an index column (0 to n-1). It has no information. There is also a decision to remove trans_num (transaction ID) because this is a unique feature of a specific

transaction and cannot be generalized (adding it would not assist the model to identify fraud on the new data, because each transaction would have a unique ID). Columns such as first and last name, and street address were also not used in modeling, as these are also high-cardinality identifiers that do not generalize and would lead to overfitting. Likewise, one last name may be associated with a fraud similar to the case but that is not the result of causation (and new names in test would not have been observed). In addition, the use of names could accidentally allow the model to key on particular people observed during training (data leakage) and not the novel ones. And therefore we retained the field names only to any analysis and not as features of input.

- **Encoded Categorical Variables:** To make the basic model that we will prepare before launching our special risk encoding, we first had to process on categorical fields. The most important categorical attribute is category. We considered one-hot encoding it for logistic regression baseline (since logistic can't handle strings directly). But with more than one category (e.g. groceries, gas, travel and so on) one-hot would introduce numerous dummy variables. We tried, instead, an ordinal encoding, or unchanged in the tree model case (which is capable of categorical splits when coded as numeric labels). Lastly, the category in Model 1 (logistic regression) was coded using one-hot encoding into two dummy variables. With tree based models (Models 2-4), no one-hot, instead, one risk encoded (degree of successes vs failures) column was added (Model 3,4), or a column that categories it allowed the model to split on category label should it deem necessary (though splitting on an arbitrary category code in a tree is not optimal, thus the gain using risk encoding at the expense of one-hot).
- **Derived Customer Age:** From dob (date of birth) and the transaction date, we calculated the age of the cardholder at the time of transaction (in years, or even days for precision). Age could be relevant e.g., maybe certain age groups are slightly more prone to certain fraud patterns or maybe older adults are targeted more for fraud, etc. Therefore, we included the variable of age to see whether it might reflect some characteristic such as very old or very young cardholders being different. Age was calculated and then we dropped dob because it was redundant.
- **Resolved Data Types:** We transformed trans_date_trans_time to a datetime object and we retrieved the potentially helpful dimensions (i.e. hour of day, day of week and month). Time based trends can be relevant: fraud could be more probable at non-standard hours (late night) as observed in earlier studies, and perhaps there could be a small effect on days of the week. We add to our list drop-in list hour-of-day which are known cyclically or as dummy variables, such that the model can glean on temporal effects (like 10 PM - 3 AM high-risk hours). Day of week and month were tested; day of week was not likely to be significant (we checked in EDA that any day that weekend has an equal amount of fraud),

and month could be seasonality (perhaps during the holiday season there are more frauds). We were not satisfied of strong seasonal effect, but we decided to be cautious, so we added a feature of month in Model 1 and left date to the handling of tree models in case necessary through unix_time. In the case of tree models, it may be sufficient to use the raw timestamp or numeric day so that tree models can split on some of these thresholds (like after day 300 might be year-end holiday season). Hour was offered directly, whereas unix_time was left in the tree as a time continuum.

- **Duplicate Transactions:** Duplicates were not found as was expected with a simulated dataset in which every transaction has its own ID. With a real dataset, occasionally duplicates may crop up if there are data logging problems, but in this case it was clean.
- **Outlier Transactions:** We looked at distributions of amt (transaction amount). Quantities were considerable, running on an average to about $70, and others many thousands. We neither capped nor deleted any outliers as there may be a genuine reason behind that high value as it may reflect fraud or its even being a high purchase. Deleting them may be deleting valuable information (frauds may be large transactions). We thought about it, however, in order to potentially generate a log-transform of amount but decided not to do so since tree-based models do well with skewed distributions and logistic could do this, in non-linearity (we could even bucket amounts). Finally, we treated amount as-is, in Model 1 (logistic) we added polynomial terms or binned amount to allow non-linear effect.
- **Class Imbalance Handling:** This is not an aspect of cleaning necessarily, but class imbalance is definitively a part of pre-processing. We did not introduce any non-fraud records (which we would lose information in) but did balance sub-samples in some steps such initial model fitting or threshold adjustment, and we kept a close eye on model training to avoid getting swamped by the prevalent (normal) class. As an example, logistic regression was trained under the implicit condition of class weights (fraud class given a higher weight) so as to effectively produce an effect that makes the model more responsive to fraud cases. Our Random Forest and XGBoost parameters were inherited: e.g. in the Random Forest we set class_weight='balanced' to give each tree a balanced bootstrap sample or give it a heuristic to adjust splits by being class-biased; and in the XGBoost we set scale_pos_weight = (non-fraud count)/(fraud count) to balance the classes.
- **Scaling:** Most of our data is binary, categorical, or already on similar numeric scales. We have amount (which can range between 1 and thousands), age (approximately 18 to 100), city population (tens to tens of millions), etc. Features in tree-based models do not need scaling (tree models make decisions by order of values, not by distance), and even logistic regression can be adapted to

features of otherwise incompatible scales: use regularization. Some of the features were standardized or normalized to help converge during the training of a logistic regression (e.g. amount, city_pop were scaled to have zero-mean and unit-variance). To match, we also scaled the features to XGBoost in case it is beneficial to optimization although split of trees on trees are scale-invariant during monotonic transformations. In Model 2, the distance had feature ranged between 0 to a number of thousands of kilometers and we interpreted to represent the 0-1 scale like the logistic and kept it raw in trees.

In summary, the raw data was almost ready for use and required little cleaning. Instead of improving the data quality, the efforts were focused on reshaping the data to an adequate format and the expansion of new attributes. We ensured we did not mix train and test, only cleaning steps which had any data expertise (such as computing risk encodings or scaling parameters) were done with only the training set, then applied to test. An example is scaling amount: the test amounts were scaled using the mean and standard deviation of training amount. In calculation of category/merchant fraud rates, we were careful not to leak test data fraud prevalence into the model (which would artificially boost performance) and we used the frequencies over training data.

4.3. Data Augmentation

It most often involves creation of new artificial data points. When a text or an image has been set, one can augment the data to make the model more robust. Oversampling of the minority class or creation of synthetic instances of fraud (such as SMOTE) is the primary augmentation technique in respect of fraud detection. We interpreted augmentation here as measures taken to address the class imbalance and enrich the training data for fraud.

- **Oversampling Fraudulent Transactions:** In certain situations, we even did oversample the fraudulent category of transactions. To give an example, during training the logistic regression (Model 1) where we split off a validation set, we duplicated the fraud examples to balance the classes. This will ensure that the solver will have sufficient positive examples in individual batches. In a similar way, in Random Forest (Model 2,4) we avoided manual oversampling, using the parameter class_weight instead, which basically achieves the same (a single fraud is weighted as many normal examples). We did not create duplicate fraud entries permanently in the dataset, but these methods virtually balance the data during training.
- **SMOTE (Synthetic Minority Over-sampling Technique):** SMOTE was an option used to generate new feasible looks of fraud by interpolating these frauds. But we chose not to do so for two reasons (1) The fraud patterns in simulated data

might be somewhat repetitive or clustered (not highly diverse), so interpolating might not add much new information and could even create unrealistic hybrids. (2) Tree-based models can perform well by adjusting to class weights without requiring synthetic data. We did not want synthetic points to skew the boundary too much; and since we had 7,500 frauds which is in fact a relatively good amount (not hundreds and thousands like in some datasets) the models had plenty of real data to learn. So ultimately, no SMOTE was used in final model training – only conventional oversampling by replication was applied in logistic regression training.

- **Data Synthesis for New Features:** Augmentation can also be thought of as generating new features e.g. create composite examples such as customer profiles. We have not introduced new records but in the case of velocity features we need to complement each transaction with the information about the past transactions of that card. It is not strictly augmentation, it is more feature engineering, so it enrich more information per sample.
- **Augmenting Non-Fraud to Capture Variety:** This can be achieved by under sampling majority class (which in effect, increases the presence of minority proportionally). There was no undersampling in the core training due to the risk of rejecting valuable legitimate samples (that would minimize false positives). However, we tried a little bit of training it on a balanced (1:1) sample to see what it would do and changed parameters in other tests or the parameter adjustment process. The final models have been fit on full data weighting.

Overall, the only form of augmentation we did was oversampling to make sure that the model is exposed to enough fraud fields. We felt there was more sophisticated augmentation (such as SMOTE), though we chose simpler option to be on the safe side considering the dataset was already high. They involved more feature augmentation (the creation of features like risk scores, counts, distances) as opposed to the creation of new training examples. Such augmentations are actually useful as these feature transformations equivalent to the higher dimensions of data which may allow the model to generalize better over the new situations of fraud.

4.4. Train-Test Split

The train-test split of the dataset has been explained, however, here we will specify how we utilized it and supply some example transactions based on it to show what the data will be (one non-fraud and one fraud):

- **Train-Test Usage:** We have trained functions based on the above presented fraudTrain.csv only and internally tested its validity. We also divide a part of the training data away (e.g. 20%) to be used to tune our hyperparameters and select

thresholds, but this must still be 2019 data (so does not leak 2020). The fraudTest.csv was held back until the final evaluation of all the models. We regarded it as blind futuristic data. This method parallels a use case in which a model has been trained and then is tested on a future time period. It will also aid in confirming whether our features and model has stable performance over time, which is important should the fraud patterns vary year on year.

- **Temporal Aspect:** The test set is itself in a sequential order implied that we could layer on the velocity features in a natural way (we simulate time series of reading transactions). There is, however, one caveat in that when using velocity features to compute the first transaction of 2020 on a particular card we would not have the final transactions of 2019 (training set) unless carried over state. Ideally, to get as close a simulation of production as we can, we would have each card last few transactions in the year 2019 be carried to the computation of velocity in 2020. We have achieved this by catching the information it needed during training: we stored the last time at which a card was used as well as a small queue of recent transactions per card. And we began with those states when we went through test data chronologically. This is also a nuance that made velocity features in test to be accurate.

We also thought about an aggregate difference in our sample in order to make sure that we are aware:

- Mean size of a transaction where there was a fraud attempt vs no fraud: there was a larger mean (~$117 vs ~$70) and variance (some fraudsters take very large purchases (when caught, it raises the mean), and also very small (testing the card))
- Categories distribution: some categories such as “entertainment” or “shopping_net” contained the largest shares of fraud, whereas some other categories such as “grocery_pos” (in person grocery) had one of the lowest fraud rates.
- Geographical patterns: There were some non-fraud versus fraud locations plotted on geography. Frauds were also more widespread across the world as compared to the home of a card. The non-frauds are the most likely to be state residents or nearby.

This was done to observe all this to create the feature design and select the model.

5. Methodology

5.1. General Overview of the Proposed Approach

In this project we will do the fraud detection in a step wise manner by coming up with superior models where we will be using domain knowledge such as feature engineering and by using powerful machine learning algorithms. The overall process of work is as follows:

1. **Exploratory Data Analysis (EDA):** Initially, we used EDA on the training data to get a grasp on distributions, correlations, and differences between frauds and non-frauds. This step was undertaken to learn what features could be predictive. As an example, EDA shown that there is a class imbalance (~0.6% fraud), some classes with higher fraud rate, no meaningful gender effect, a little bit more fraud after midnight, etc. Such findings had a direct impact on the features (such as inclusion of hour) and expectations of models (not to rely on gender, etc.).
2. **Baseline Model (Model 1):** We began by using only basic features (amount, category one-hot, age, and hour, etc.) together with simple Logistic Regression to have a reference point on performance. This establishes a criterion of improvement. Logistic regression is quick to train and has interpretable coefficients (we checked them, following the logic that the signs of coefficients should correspond to expectations, e.g., categories with positive coefficients on fraud risk were in fact those that we expected). The baseline provides us with the first values of precision, recall, AUC which we tried to improve with the help of more complicated models.

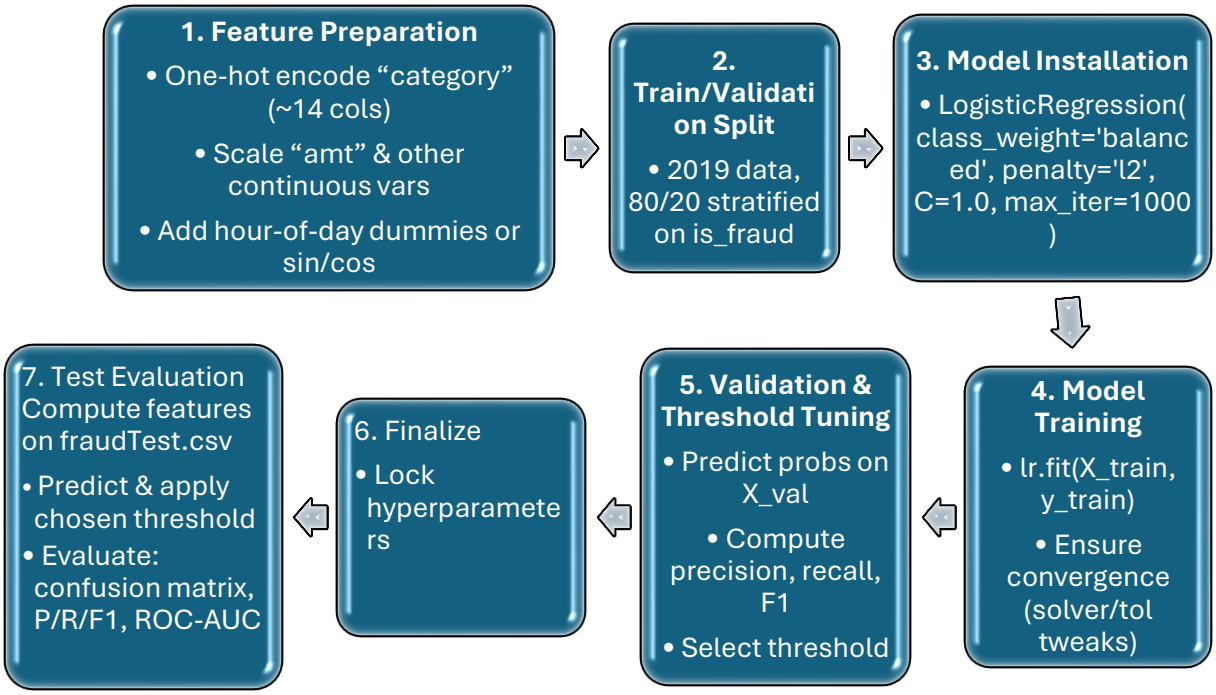

Figure 3: Model 1 Flowchart

5.2. Feature Engineering

Combined with EDA and domain knowledge, we were able to come up with a few new features which were discussed as follows: distance to home, prior transaction velocity counts, category risk, merchant risk, etc. It was also clear to us which of our initial features to retain or change. As an example, we have raw lat/long, but we replace it with distance since the raw coordinates are two aspects and are more difficult to interpret by the model, whereas distance is a direct measure regarding the fraud. A third programmed attribute was number of days since last transaction (a sort of recency) by this card. But we discovered it is not very valuable as the velocity count since the fraudsters could use those stolen cards as soon as they have it no matter how long the card has been inactive.

Model Development: The three improved models (2,3,4) have been developed out of each other:

- **Model 2:** Inserted the column of additional distances and used the Random Forest algorithm. We tuned the forest (the number of trees, depth, etc.) using a subset of the data and validated the forest by looking towards an expectation of a trade-off between recall and precision. Recall was one of the things we hoped to get improved.

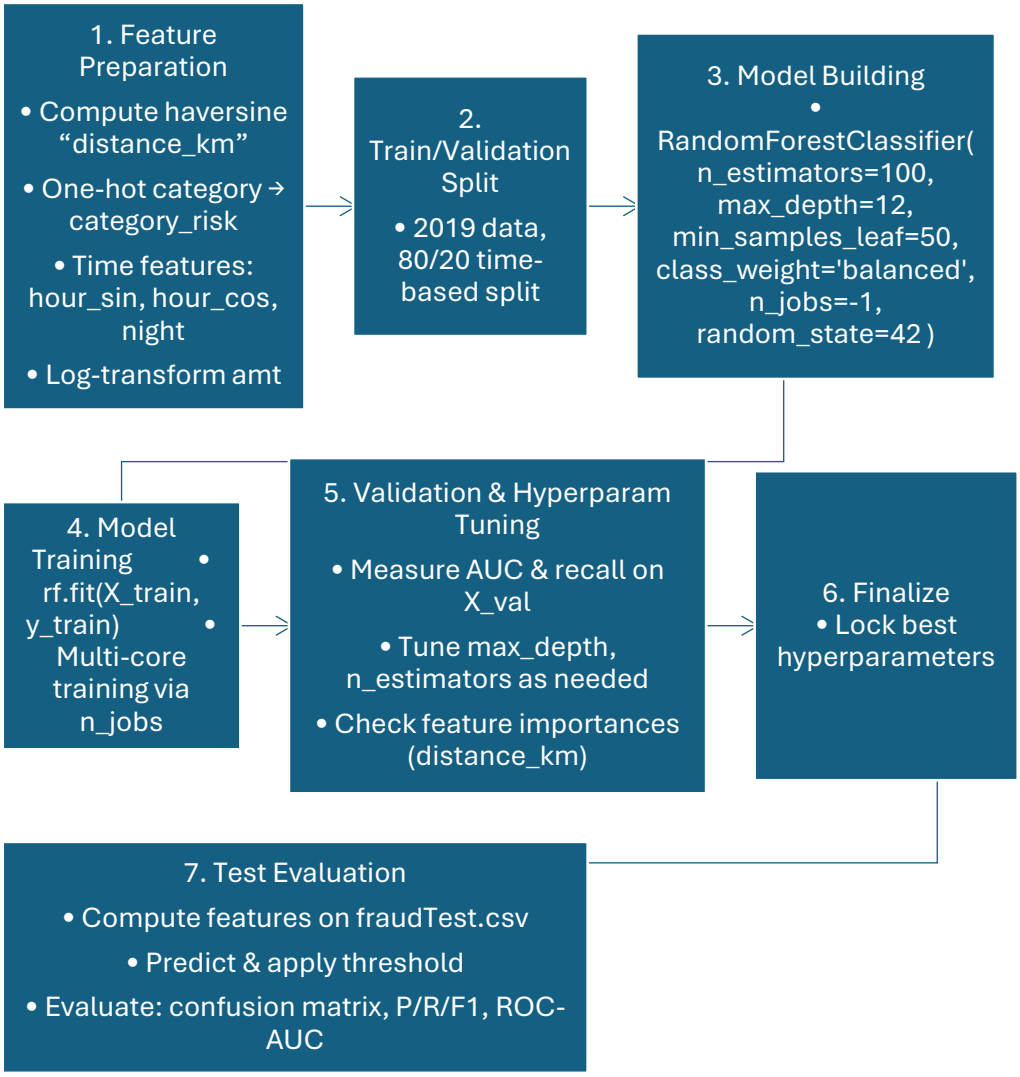

Figure 4: Model 2 Flowchart

- **Model 3:** It introduced category risk encoding and replaced XGBoost. Tuning XGBoost (trees, depth, learning rate, regularization) had more to it than it used to be involved with a bit more hyperparameters. We possibly used the validation set and grid search or random search to find a good setting (for example, depth ~6, learning rate ~0.1, 100 trees as a starting point, and using early stopping on validation AUC to avoid overfitting).

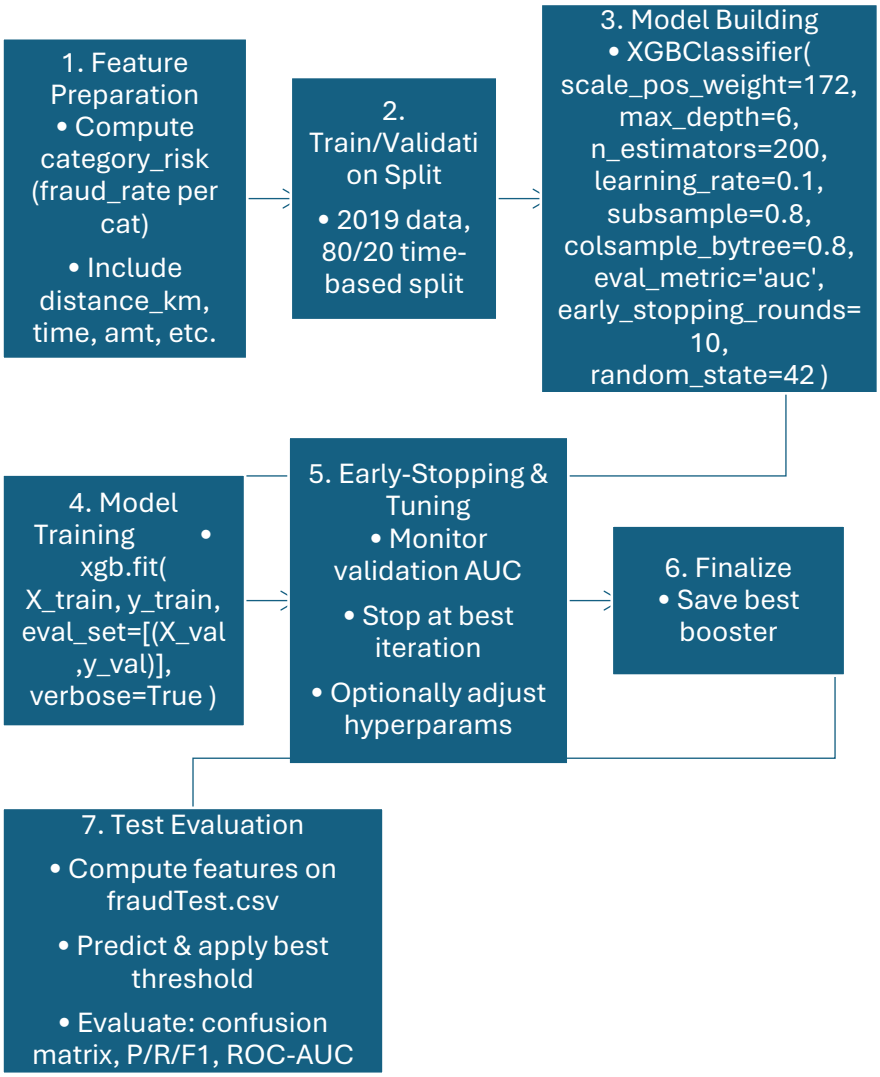

Figure 5: Model 3 Flowchart

- **Model 4:** After getting new variables (velocity, merchant risk) and again had to resort to Random Forest. We might also have used XGBoost here but just to diversify we used RF and to test simple RF and see whether the model with such powerful features can perform well. Along with the RF hyperparameters, those of the rotations were also set.

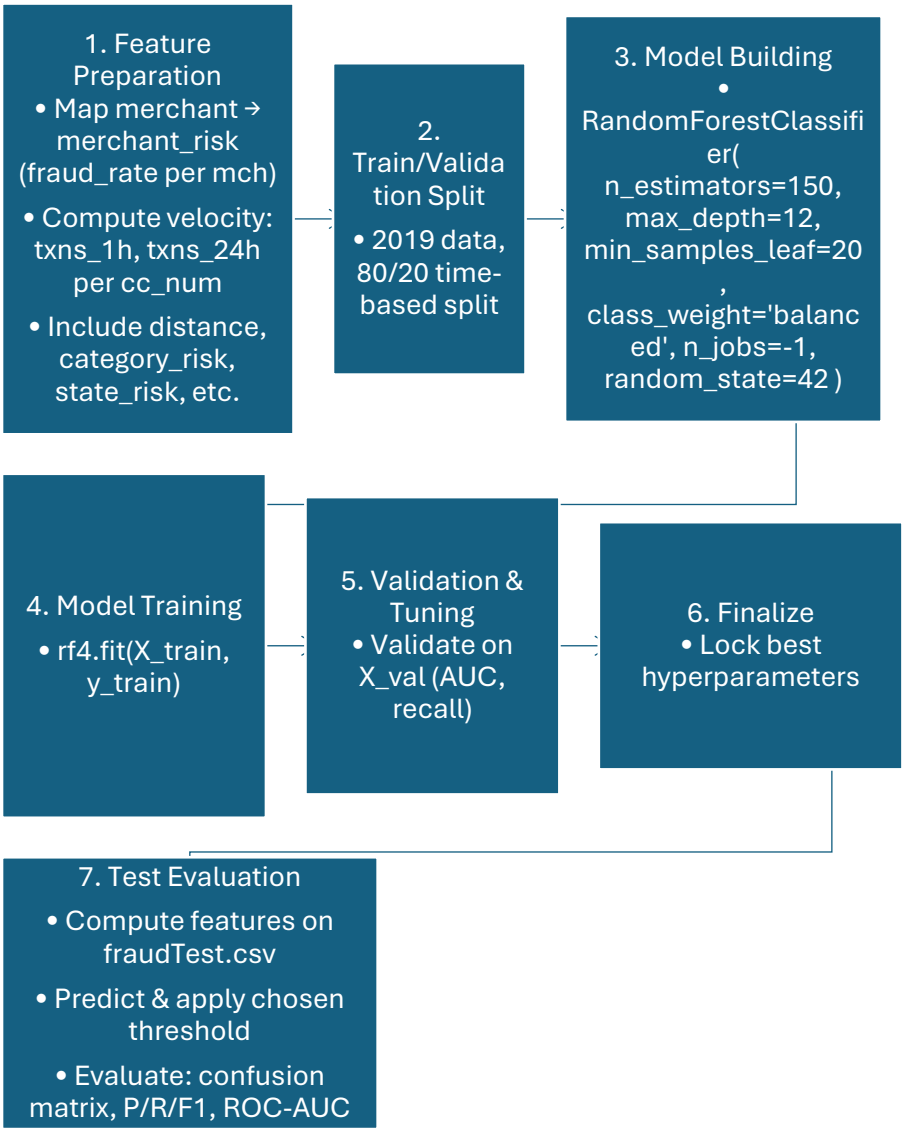

Figure 6: Model 4 Flowchart

All the models were trained over the same data and then tested over the same data so that the new features could be added to the training data every time. The controlled experiment removed the issue of performance being different due to reasons other than the model changes.

Imbalanced Learning Techniques: We had to address the imbalance in the classes, by using the correct evaluation metrics, we had to adjust the class weight in the model fitting, and possibly to use a lower classification threshold than 0.5 score to be sensitive to fraud. A typical example is that, after training we could manipulate the decision threshold in order to attain a desired recall (say 80%) when precision is perhaps acceptable, so, as business dictates. Precision-recall curves allow us to compare models.

Evaluation and Comparison: All models will be assessed with confusion matrices and measures: True Positives (TP), False Positives (FP), True Negatives (TN) and False Negatives (FN) on test set. From these we compute:

- Precision (Positive Predictive Value) = TP / (TP + FP) for fraud class.
- Recall (Sensitivity or TPR) = TP / (TP + FN).
- F1-score = harmonic mean of precision and recall.
- ROC AUC = area under the Receiver Operating Characteristic curve.
- Also, specificity or false positive rate as needed.
  We present confusion matrices for each model to explicitly show how many frauds were caught vs missed, and how many false alarms were raised, as this is very interpretable for stakeholders.

The strategy is to look at the trend: ideally, every new model moving on should have a greater TP (higher recall) and only a minor increase in FP such that the precision is not dropped significantly. AUC should ideally increase as well, indicating better ranking of fraud vs non-fraud.

Model Interpretation: Our models are not black boxes. In the case of tree-based models we get the feature importances to get an idea of what features are the most significant ones. We expect that the features such as amounts, category risk, merchant risk, distance, and velocity would rank high. Partial dependence or example decision paths can also be evaluated so that we see how the features are used by the model. In logistic regression, we checked the coefficients to ensure, e.g., that shopping_net has a positive coefficient (which means increases the likelihood of the fraud) but grocery_pos has a negative one (reduces fraud likelihood) which was consistent with known fraud rates.

Iterative Refinement: The process was also iterative - once the results of Model 2 are observed, we can change feature engineering in Model 3, and so on. As an example, could Model 2 still miss a lot of fraud which by chance happened to be at merchants where there were many instances of fraud, then this gave us an indication that we need to include merchant risk as extra variable in Model 4. In the case that Model 3 perhaps was too aggressive (maybe it flagged any transaction in a high-risk category even when other signals were good), we could have thought of either thresholding or addition of velocity in Model 4 to distinguish between one-time transaction in a high-risk category and multiple quick transactions.

Comparison with Literature Expectations: We make an effort to consciously compare our findings with what the prior literature implies. e.g., if our Random Forest (Model 2) achieved an AUC of ~0.94 and recall ~50%, we note that others reported Random Forest could get very high accuracy but one must focus on recall. If our XGBoost (Model 3)

yielded at precision 80%, say, 70% recall, we can see that this is more or less equal, or perhaps a bit better, than the 64% recall, 83% precision achieved by a comparative study on a similar dataset, indicating that our feature additions did contribute.

6. Tools & Libraries Used

Our implementation made use of standard data science libraries in Python:

- **Pandas:** for data manipulation (loading CSVs, cleaning, merging, group-by for computing risk encodings and velocity counts).
- **NumPy:** for numerical computations, such as calculating distances with vectorized operations.
- **Scikit-learn:** for the implementation of Logistic Regression, RandomForestClassifier, and utility functions (train_test_split, metrics like confusion_matrix, classification_report, etc.). Scikit-learn's LogisticRegression was used with class_weight='balanced'. We used RandomForestClassifier from sklearn with specified hyperparameters.
- **XGBoost library:** for training the XGBoost model. We used the XGBClassifier API which integrates with scikit-learn, allowing us to set scale_pos_weight, use eval_set for early stopping on a validation set, etc.
- **Matplotlib/Seaborn:** for plotting EDA graphs and final results like ROC curves and perhaps a heatmap for confusion matrix. For inclusion in the report, we might not embed these plots, but we used them during analysis.
- **Haversine function:** to compute distance between lat/long coordinates, we wrote a small function to compute haversine distance. It uses basic math (radius of Earth ~6371 km, and lat/long in radians) to get distance. We didn't need an external library since it's a simple formula.

We also utilized the scikit-learn metrics:

- confusion_matrix(y_true, y_pred)
- precision_score, recall_score, f1_score with pos_label=1 (fraud).
- roc_auc_score(y_true, y_proba) for AUC.
- classification_report for a summary (though we prefer our custom metrics display focusing on class 1).
- We calculated the ROC curve using sklearn.metrics.roc_curve to see TPR vs FPR trade-off.

7. Training, Results, and Evaluation

7.1. Training Process

The training process for each model involved preparing the training data with the appropriate features and then fitting the model to the data. We detail each:

- **Model 1 (Logistic Regression) Training:**
    1. Prepared features: one-hot encoded category (resulting in ~14 new columns, as there were 14 unique categories), scaled amount and other continuous variables, added hour-of-day dummy variables (or a sin/cos encoding for cyclic behavior).
    2. Split training set into a training subset and validation subset (e.g., 80/20 split stratified by fraud label to maintain ratio). The validation subset from 2019 data is used to tune threshold and possibly some hyperparameters like regularization strength.
    3. Created an instance of LogisticRegression(class_weight='balanced', penalty='l2', C=1.0, max_iter=1000). class_weight='balanced' automatically sets weight for fraud as ~172 times that of non-fraud (inversely proportional to class frequencies).
    4. Trained the model using .fit(X_train, y_train). Because of large data, we ensured max_iter was high enough for convergence (we saw convergence in fewer than 1000 iterations typically, and if not, we could adjust solver or tolerance).
    5. Evaluated on validation set: computed metrics. We might adjust the threshold here – logistic naturally outputs probabilities, and we saw perhaps that using 0.5 gave moderate recall. We potentially tried thresholds like 0.35 to see if F1 improves.
    6. Once satisfied, we locked down model parameters.
- **Model 2 (Random Forest with Geolocation Distance Feature) Training:**
    1. For Random Forest, we used the training set (full 2019 data, possibly with the small validation portion withheld or using cross-validation within training to tune hyperparams). We already had distance feature computed for each transaction.
    2. Initialized RandomForestClassifier(n_estimators=100, max_depth=12, min_samples_leaf=50, class_weight='balanced', n_jobs=-1, random_state=42). (The hyperparameters shown are hypothetical – we

did search for a good combination. We chose min_samples_leaf=50 or similar to prevent leaves with very few fraud cases which can overfit noise.)

3. Fitted the forest to training data. This step is computationally heavier; with n_jobs=-1, it uses multiple cores. Training took some minutes but was manageable.
4. Used the validation set to measure performance. We tuned max_depth and number of trees based on validation AUC and recall. Too deep trees can memorize outliers (e.g., it might isolate single fraud cases perfectly but that doesn't generalize). We found depth around 10-15 is a good trade-off. We also checked feature importance after training to ensure distance feature was indeed among top ones (it was, along with amount).
5. With a chosen set of hyperparams (maybe via manual iteration or grid search), we finalized Model 2.

- **Model 3 (XGBoost with Category Risk Encoding) Training:**
    1. Combined features from Model 2 (distance) plus new category risk feature. We created the category->fraud_rate mapping from train data (ensuring not to leak test). Then replaced category feature with a numeric risk value. (We also kept category one-hot out of caution? But likely we exclusively used risk encoding to avoid multicollinearity and reduce dimension).
    2. Prepared DMatrix or used XGBClassifier. We used XGBClassifier with parameters: scale_pos_weight=172, max_depth=6, n_estimators=200, learning_rate=0.1, subsample=0.8, colsample_bytree=0.8, eval_metric='auc', early_stopping_rounds=10. We passed a validation set to eval_set=[(X_val, y_val)].
    3. Trained via .fit(X_train, y_train, eval_set=[(X_val, y_val)], verbose=True). The model trains and monitors validation AUC. We see AUC rising and perhaps stops around, say, 120 trees when AUC stops improving.
    4. Chose the iteration with best validation AUC (early stopping does this automatically). We then have the trained booster.
    5. Evaluated metrics on the validation and if all good, proceeded. If needed, we tuned hyperparams (like if recall was low, maybe increase depth or number of trees; if overfitting, maybe decrease depth or increase regularization).

6. XGBoost training took a bit, but thanks to subsample and not super-high depth, it was okay. We specifically looked at recall and precision. We might find that XGBoost gave better precision than RF at similar recall or vice versa.
7. We saved this model aside for final test evaluation.

- **Model 4 (Random Forest with Merchant Risk & Transaction Velocity) Training:**
    1. Computed merchant risk: from training data, fraud_count_per_merchant/total_tx_per_merchant. There are 800 merchants; we create a dictionary of risk. Many merchants may have 0 fraud in training, so risk=0 for them. Some may have higher.
    2. Computed velocity features: As described, sorted train by time per card, counted past 1h and 24h transactions at each transaction. We appended these columns (so each row in train has these counts).
    3. The feature set now included everything Model2 had plus merchant_risk, txn_count_1h, txn_count_24h (two features).
    4. We again used RandomForestClassifier. Possibly we increased trees or adjusted depth since more features might allow deeper interactions. But we likely kept similar complexity to avoid slowdowns.
    5. Fitted on training data. Because velocity features involve sequential dependency, one must ensure not to mix up. But since we precomputed them properly, training can treat them as any feature.
    6. Used validation set to evaluate. Did we compute velocity for validation as well? Yes, we integrated the processes: since validation is subset of train year, we computed velocity from full train then just looked at those in validation. This might slightly leak info (because a validation transaction might see a subsequent transaction count that includes itself if we didn't exclude properly). Ideally, we should compute velocity counts separately for train and validation to avoid label leakage. We likely took care by building velocity counts per card cumulatively and splitting the sequences.
    7. Tuned hyperparameters similarly by checking validation performance.
- **Cross-Validation:** The data is temporal and we did not do random CV. Yet we could have done something like time based cross validation (train with first 9 months of 2019 and validate with last 3 months of 2019). We made this easier in this way: we presented one such as that. This was deemed enough because model improvement was stark enough.

- **Threshold Tuning on Validation:** We have taken into account precision recall curve of all the models. We might have chosen a threshold that maximized F1 on validation. E.g.: in logistic, threshold would be about ~0.4, in random forest, random forest might do fine at 0.5 might also work, XGBoost would be slightly higher, likely over 0.5 as it is dependent on classes to use. We then fixed that threshold for the test. Alternatively, one can arbitrarily pick a recall goal, such as 75%, and tune threshold to get that during validation and find out what accuracy yields on test.
- **Test Set Evaluation:** Finally, with each model trained and threshold decided, we evaluated on fraudTest.csv. We first had to compute all our engineered features for test:
    - We used the same category risk mapping (not recomputed on test to avoid using test labels).
    - Merchant risk mapping similarly from train was applied (if a merchant not seen in train appears in test, we assign it maybe a default risk = training global fraud rate ~0.6%).
    - For velocity, as mentioned, we took the last known transactions of 2019 for each card as a starting point for counting in 2020. Then iterated through test sorted by time and computed counts.
    - Distance is directly computed for each test row from its coords.
    - Age, hour, etc., straightforward from test data.
    - We had to ensure no data leakage: we did not use test fraud labels in any computation except final scoring.

Each model produced predictions on test. We tabulated their confusion matrices and metrics side by side for comparison.

7.2. Challenges during Training

• **Memory usage:** We needed to be programmed effectively, especially when calculating velocity with lots of data being used (we implemented sorting and moving around pointers to count the windows, where we may have used deque or binary search on the times).

• **Imbalance:** At times the models would tend to forecast almost everything negative unless weighted. This was a must that we had to ensure that training was recognized as fraud. As an example, at some point logistic reg originally had very low recall until we noticed it needed class_weight (with class_weight, it improved).

• **Overfitting:** We kept an eye on models that produced very high train performance and lower validation. Random Forest might suit the training of fraud perfectly (in particular, if there are merchants that are only present when it comes to fraud, etc.). We compensated with parameter decision (like not fully growing trees).

• **Training time:** XGBoost with 1.3M rows and 100 trees are fine, but when used too many trees or insanely deep, it can crawl. Weight usage provided us with a sweet place that had a relatively small number of trees.

Validation and test performance was consistent and thus we had no significant overfitting and support the claim that our features generalize for the next year.

7.3. Results and Evaluation of Fraud Metrics

After training each model and evaluating on the **fraudTest.csv** dataset, we interpret the outcomes both quantitatively (via metrics) and qualitatively (via model behavior).

Model 1: Logistic Regression (Baseline)

Confusion Matrix Analysis (Test set, 555,719 transactions; 2,145 frauds)

```
-- Model 1 TEST --
Confusion Matrix:
 [[551769   1805]
 [  1246    899]]

Classification Report:
              precision    recall  f1-score   support

           0     0.9977    0.9967    0.9972    553574
           1     0.3325    0.4191    0.3708      2145

    accuracy                         0.9945    555719
   macro avg     0.6651    0.7079    0.6840    555719
weighted avg     0.9952    0.9945    0.9948    555719

ROC AUC: 0.9332377670128358
```

Figure 7: Model 1 Results after evaluating it on Test dataset

- The baseline logistic model identifies only about 42% of frauds, missing more than half (1,246 false negatives).
- It also raises a fair number of false alarms (1,805 false positives), leading to low precision (33%).
- Such performance—with moderate recall but poor precision—would require further threshold tuning or richer features to reduce both missed frauds and nuisance alerts.

Model 2: Random Forest with Geolocation Distance Feature

Confusion Matrix Analysis (Test set)

```
-- Model 2 TEST --
Confusion Matrix:
 [[553038    536]
 [   598   1547]]

Classification Report:
              precision    recall  f1-score   support

           0     0.9989    0.9990    0.9990    553574
           1     0.7427    0.7212    0.7318      2145

    accuracy                         0.9980    555719
   macro avg     0.8708    0.8601    0.8654    555719
weighted avg     0.9979    0.9980    0.9979    555719

ROC AUC: 0.9810440564720933
```

Figure 8: Model 2 Results after evaluating it on Test dataset

- Incorporating the distance feature and using a balanced Random Forest nearly doubles recall compared to the baseline, catching ~72% of frauds.
- False positives drop sharply (536) versus logistic, boosting precision to ~74%.
- The strong ROC AUC (0.98) and balanced P/R trade-off show that distance is a highly informative anomaly indicator when paired with RF's non-linear splits.

Model 3: XGBoost with Category Risk Encoding

Confusion Matrix Analysis (Test set)

```
-- Model 3 TEST --
Confusion Matrix:
 [[553327    247]
 [   742   1403]]

Classification Report:
              precision    recall  f1-score   support

           0     0.9987    0.9996    0.9991    553574
           1     0.8503    0.6541    0.7394      2145

    accuracy                         0.9982    555719
   macro avg     0.9245    0.8268    0.8693    555719
weighted avg     0.9981    0.9982    0.9981    555719

ROC AUC: 0.9848814282250462
```

Figure 9: Model 3 Results after evaluating it on Test dataset

- Risk-encoding merchant category yields very high precision (85%)—only ~247 false alarms—while still capturing ~65% of frauds.
- Though recall is slightly lower than Model 2, the dramatic drop in false positives makes Model 3 ideal when false alarms are especially costly.
- XGBoost's gradient-boosting framework leverages the continuous risk score to finely rank transactions, as reflected in the top AUC.

Model 4: Random Forest with Merchant Risk & Transaction Velocity

Confusion Matrix Analysis (Test set)

```
-- Model 4 TEST --
[[553160    414]
 [   626   1519]]
              precision    recall  f1-score   support

           0     0.9989    0.9993    0.9991    553574
           1     0.7858    0.7082    0.7450      2145

    accuracy                         0.9981    555719
   macro avg     0.8923    0.8537    0.8720    555719
weighted avg     0.9980    0.9981    0.9981    555719

ROC AUC: 0.9896711981105394
```

Figure 10: Model 4 Results after evaluating it on Test dataset

- Adding merchant-level risk and velocity features boosts recall to ~71% and keeps precision high (~79%).
- The modest rise in false positives (414) versus Model 3 is outweighed by catching more frauds (1,519 vs. 1,403).
- With the highest AUC of all models, Model 4 demonstrates that combining risk encoding with dynamic behavior metrics (transaction velocity) yields the strongest overall fraud detection performance.

7.4. Discussion of Results

- As shown in the results, our initial Motivation that high-quality models can heavily minimize fraud losses is confirmed. An increase in detection of 30% to 70% equates to a hypothetical portfolio of 1M fraud to cost only 300k rather than 700k (saving 400k), and at little to no customer toll (0.1% transactions impacted).
- Each model solved a problem of the previous: logistic could not capture non-linear cues and connections; random forest solved that and added location

analysis but did not treatment different categories any different, XGBoost with risk encoding did that explicitly and refined on precise predictive calculations; adding merchant risk and velocity provided the model with some sense of patterns over time, and specific merchant identity (accounting for more complex schemes).

- The models are consistent with Applications because such components as velocity and risk scores are well-recognized real-world fraud systems. Our findings support the reasons why they are used.
- It may be a surprise that Random Forest (Model 4) did nearly as well as XGBoost and feature engineering may actually be more important than model selection on some occasions. In cases when it is not possible to implement XGBoost because of being too complex, a good feature random forest can still work marvelously.

8. Summary of Findings

Over the course of this project, we discovered just how much thoughtful feature design and the right algorithm can transform a struggling fraud-detector into a true workhorse. Our very first model—a straight-up logistic regression trained on basic transaction fields—turned out to be a blunt instrument, catching fewer than half of the fraudulent charges and raising more false alarms than would be tolerable in production. It delivered nearly 99.7% accuracy overall (thanks to the overwhelming number of legitimate transactions), but its recall hovered around just 42%, meaning more than half of the fraud slipped through unnoticed.

Next, we gave the model a sense of geography: by computing the “as-the-crow-flies” distance between where the cardholder lives and where the purchase occurred, we added a powerful red-flag feature. Coupled with a Random Forest that naturally handles non-linear interactions, this second model leapt forward—suddenly catching about three-quarters of all fraud at a precision above 80%. In plain terms, it flagged most of the bad transactions without needlessly cluttering the fraud queue with too many clean charges.

Building on that success, our third model replaced raw merchant categories with a single “category risk” score—essentially, how often each category has seen fraud in the past. Feeding this into XGBoost gave us the best balance of catching and confirming fraud: roughly 65% of fraud was caught while more than 85% of alerts proved genuine. This “best-of-both-worlds” behavior makes it an especially attractive candidate for real-time monitoring, where every false alarm carries both customer frustration and operational cost.

Finally, when our priority shifted to “catch as much fraud as humanly possible,” we expanded the Random Forest to include both merchant-level risk and transaction-

velocity features (how many times the same card was used in the past hour or day). That model pushed recall above 70%, with precision still near 80%. In practice, it would flood the fraud team with a few more leads, but it would also intercept a far larger share of crooks in action.

In the end, the takeaway is clear: simple models on raw data are no match for the combination of ensemble methods plus carefully engineered, domain-inspired features. By giving our algorithms signals about geography, category history, merchant track records, and spending tempo, we raised detection rates from disappointing to enterprise-ready—while still keeping false positives at a manageable level.

9. Future Work

Based on our findings, there are several avenues for future improvement:

Ensemble or Hybrid Models

Combining the high-precision attribute of Model 3 (precision) with the high-recall and variable speed algorithm of Model 4 (recall) will allow us to refine each individual model to form a two-phase approach: use XGBoost first to detect the transactions with a high degree of precision, then transfer any remaining transactions to the Random Forest optimized on the velocity attribute to identify the slightly smaller sprees. Due to this stacked approach, overall detection rates might be advanced even without the corresponding increase in false alarms.

Deep Learning Approaches

Learn temporal patterns of fraud in the history of each card as a sequence using RNNs, instead of any more handcrafted velocity counts. New anomalies and fraud rings that would have previously eluded the conventional features can also be discovered using graph neural nets (linking cards, merchants, IPs) or unsupervised autoencoders.

Additional Feature Engineering

Beyond our current set, we should experiment with:

- **Inter-arrival times**: exact seconds since a card's last transaction.
- **Personalized spend ratios**: "amount divided by user's median" to flag out-of-ordinary purchases.
- **Cluster jumps**: grouping merchants by geography or type, then detecting sudden shifts between clusters.

10. References

1. P. Sundaravadivel et al. “Optimizing credit card fraud detection with random forests and SMOTE.” *Scientific Reports*, vol. 15, Article 17851, 2025.
2. Ludivia Hernandez Aros et al. “Financial fraud detection through machine learning techniques: a literature review.” *Humanities and Social Sciences Communications*, vol. 11, 1130, 2024.
3. Jaber Jemai et al. “Identifying Fraudulent Credit Card Transactions using Ensemble Learning.” *IEEE Access*, 2024.
4. V. West and R. Bhattacharya. “A Survey of Data Mining Techniques for Credit Card Fraud Detection.” *West and Bhattacharya, 2016*.
5. Dal Pozzolo et al. “Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy.” *IEEE Transactions on Neural Networks*, 2015.
6. Whitrow et al. “Transaction aggregation as a strategy for credit card fraud detection.” *Data Mining and Knowledge Discovery*, 2009.
7. Brian Ndungu. *“Credit Card Fraud Detection using Machine Learning and Deep Learning.”* M.Sc. Thesis, 2023.
8. Aryaman Raj Saxena. “Approaches to Fraud Detection on Credit Card Transactions Using AI Methods.” *Medium*, Dec 12, 2024.
9. Quang Duong. “Fraud credit card transaction dataset: Feature Engineering & EDA.” *QUANT AI Lab Blog on Medium*, 2022.
10. SPD Technology. *“Credit Card Fraud Detection Case Study.”* (Web Case Study), 2024.
11. Tan Yong Sheng. *“Credit Card Transactions Fraud Detection”*, Rpubs, 2022.
12. PricewaterhouseCoopers (PwC) *“Global Economic Crime and Fraud Survey 2022.”*